Mobile Application Development

Course Outline

Duration: 1 Year

Modality: Hybrid (Online & In-Person)

Technologies: JavaScript, NodeJS, MongoDb, Firebase, Artificial Intelligence (*Integration in Mobile Development*), Flutter, Dart, NodeJS

Focus Areas: Programming Fundamentals, Backend Development, Frontend Development, AI Concepts and Integration, System Design, Monetization Strategies, Business Considerations

Module 1: Programming Fundamentals

- **Introduction to Programming**
  - Understanding programming logic & problem-solving
  - Algorithms and flowcharts
  - Pseudocode and coding best practices
- **Introduction to Java**
  - Data types, variables, and operators
  - Control structures: if-else, loops (for, while, do-while)
  - Functions and modular programming
  - Debugging and error handling
- **Object-Oriented Programming (OOP)**
  - Classes, objects, and methods
  - Inheritance, polymorphism, encapsulation, abstraction
  - Interfaces and abstract classes
  - Exception handling

Module 2: Flutter Development

Part 1: Dart Programming

- **Introduction to Dart**
  - Data types, variables, and operators
  - Control structures: Loops, Conditional Statements
  - Functions, Classes, and Objects
  - Exception Handling
- **Collections in Dart**
  - Lists, Sets, Maps, Queues
- **Asynchronous Programming**
  - Future, Async/Await, Streams

Part 2: Flutter Framework

- **Mobile App Security**
  - Secure coding principles
  - API security and token-based authentication
  - Data encryption techniques
- **Flutter Setup and Basics**
  - Installing Flutter and setting up IDE
  - Flutter Project structure
  - Widgets, Stateless vs. Stateful Widgets
  - Handling gestures and navigation
- **Building UI with Flutter**
  - Layout Widgets (Column, Row, Stack, Flex)
  - Forms and validation
  - State Management (Provider, Riverpod, Bloc)
  - **User Experience (UX) Design**
    - Wireframing and prototyping
    - Accessibility and usability testing

- **Networking & API Integration**
    - Making HTTP requests with Dio
    - Parsing JSON data
    - Secure API handling
- **State Management & Storage**
    - **Introduction to State Management**
        - Why is state management important?
        - Overview of different approaches: BLoC, Provider, Riverpod, GetX
        - Examinaing State Management Providers
        - **Provider** – Simpler, reactive programming
        - **Riverpod** – More scalable, eliminates dependency context issues
    - **BLoC (Business Logic Component)**
        - Understanding Events, States, and Cubit
        - Implementing BLoC in a real-world app
        - Best practices for large-scale applications
    - **Local Storage in Flutter**
        - SharedPreferences vs Hive vs SQLite – Choosing the right storage
        - Implementing **Hive for lightweight data persistence**
        - Using **sqflite for structured relational data**
- **Cloud Storage in Flutter using Firebase Integration**
        - Firebase Basics
        - Firebase Authentication
        - Push Notifications

- **Flutter Animations & Performance Optimization**
    - AnimatedBuilder, Hero Animations
    - Lazy loading and image caching
- **Platform-Specific Code**
    - Handling iOS and Android-specific functionalities
    - Accessing device sensors and services

Module 3: MongoDB & Mongoose for API and Mobile Development

Advanced JavaScript Concepts

- Modern JavaScript (ES6+) essentials
- Callbacks, Promises, and async/await
- Error handling patterns
- Working with JSON
- Modules and clean folder structure

- **Introduction to Databases**
  - **Understanding Databases & Data Storage**
    - What is a database?
    - Why databases matter for mobile and web apps
    - Differences between SQL and NoSQL
    - Real-world use cases (social media, e-commerce, chat apps, etc.)
- **MongoDB (NoSQL) – Core Concepts**
  MongoDB Architecture
  - Collections vs Tables
  - Documents vs Rows
  - BSON vs JSON
  - Schemaless data design (and schema flexibility)

  Document Modeling
  - One-to-One, One-to-Many, Many-to-Many in MongoDB
  - Embedded documents vs referencing
  - Designing scalable document relationships

- **Mongoose ORM/ODM**
  Schema & Model Design
    - Creating schemas
    - Creating and exporting models
    - Data types & validation
    - Timestamps
    - Virtuals and middleware
    - Indexes and unique constraints

  CRUD with Mongoose
    - Creating documents
    - Finding and querying (find, findOne, regex)
    - Updating (updateOne, findByIdAndUpdate)
    - Deleting documents
    - Pagination and filtering
- **Querying & Indexing in Mongoose**
    - Indexes (single, compound, text search)
    - Performance optimization
    - Projections (select fields)
    - Aggregation pipelines (grouping, lookups, analytics)
- **NoSQL Ecosystem Overview**

  Types of NoSQL Databases
    - **Document-based:** MongoDB, Firebase Firestore
    - **Key-Value Stores:** Redis
    - **Column Stores:** Cassandra
    - **Graph Databases:** Neo4j

  When to Use NoSQL vs SQL
    - Flexible vs strict schema
    - Scalability considerations
    - Real-time requirements
    - Mobile-friendly offline support

Module 4: Backend Development with Node.js + Express.js

- **Introduction to Node.js**
  - What is Node.js?
  - Event-driven architecture
  - Non-blocking I/O
  - Package management with npm
  - Setting up a NodeJs project using **npm init**
- **Introduction to Express.js**
  - Creating an Express app
  - Understanding HTTP requests and responses
  - Understanding middleware
  - Using routers
  - Environment variables (dotenv)
  - API folder structure best practice
- **Building RESTful APIs**

  Controllers & Routes
  - Creating routes with GET/POST/PUT/DELETE
  - Request & response lifecycle
  - Query params, body params, and route params
  - Modular router structure

  Validation & Error Handling)
  - Input validation with Joi / express-validator
  - Global error handler middleware
  - Sending proper HTTP status codes

- **Authentication & Security**

  JWT Authentication

  - Generating JWT tokens
  - Protecting routes (auth middleware)
  - Refresh tokens
  - Password hashing (bcryptjs)

  Role-Based Access Control (RBAC)

  - User roles and permissions
  - Middleware-based role restrictions

  OAuth2 & Social Logins

  - Google login
  - GitHub Login
  - Token exchange and verification

  Other Security Essentials

  - CORS
  - Helmet
  - Rate limiting
  - Sanitization

- **Database Integration with MongoDB + Mongoose**
  - Connecting Node.js to MongoDB
  - Creating models and schemas
  - Performing CRUD operations
  - Hooks, validators, indexes
  - Aggregation pipelines
  - Transactions (MongoDB Sessions)

- **API Testing**
  - Postman
  - Writing basic tests with Jest

- **Building a Full-Stack Application**
  - Connecting Node.js backend to Android/Flutter
  - API response formats (JSON standards)
  - Image uploads (Multer, Express-File-Upload)
  - Push notifications via backend
  - Deploying backend (Render, Railway, AWS, DigitalOcean, GoDaddy)

Module 5: Introduction to AI and AI Integration in Mobile Development

1. Introduction to Artificial Intelligence (AI)

- What is AI? Understanding **Machine Learning (ML)** and **Deep Learning**
- How AI is transforming mobile applications
- Real-world AI use cases in mobile development:
    - **Automation** (e.g., chatbots, smart assistants)
    - **Personalization** (e.g., recommendation systems)
    - **Security** (e.g., facial recognition, fraud detection)

2. Prompt Engineering for AI-Powered Apps

- Basics of AI prompting: **Structured vs. Open-ended prompts**
- Crafting **effective prompts** for AI-powered chatbots and assistants
- Fine-tuning AI responses for **Natural Language Processing (NLP)**
- AI-generated content in mobile apps (text, images, voice)

3. Introduction to AI Tools & APIs for Mobile Development

- **OpenAI (GPT models)** and **Google Bard** for chat features
- **AI-powered voice recognition & speech-to-text**
    - Google Speech-to-Text API, Deepgram, Whisper
- **ML Kit (Flutterfire)**
    - Image Labeling, Face detection, OCR (Text recognition)
- **Recommendation Basics**
    - Build a simple collaborative filter using:
        - API scoring
        - Similarity comparison

4. Deployment & Optimization of AI-powered Mobile Apps (Basics Only)

- Considerations for **AI computation on-device vs. cloud-based solutions**
- Performance considerations:
    - Battery and memory usage for AI models
    - Handling large datasets and real-time AI processing

5. Ethical considerations & responsible AI practices:

- AI bias & fairness in mobile applications
- Privacy concerns and local DPA compliance
- Secure handling of AI-generated data

Module 6: System Design and Business Considerations

- **Software Development Lifecycle (SDLC)**
  - Agile vs Waterfall methodologies
  - Requirement gathering and documentation
  - Wireframing and prototyping tools
- **System Design for Mobile Applications**
  - Monolithic vs Microservices Architecture
  - Scalability and security considerations
  - API design principles
- **Performance Optimization**
  - Efficient memory management
  - App profiling and debugging tools
  - Caching strategies
- **Performance Optimization**
  - Efficient memory management
  - App profiling and debugging tools
  - Caching strategies
- **User Experience (UX) Design**
  - Wireframing and prototyping
  - Accessibility and usability testing
- **Testing Strategies**
  - Unit testing with JUnit
  - UI Testing with Espresso (Android) & Flutter Test
  - Integration testing for APIs
- **CI/CD for Mobile Apps**
  - Version control with Git/GitHub
  - Automated testing and debugging
  - Deployment strategies (Google Play, App Store)

- **Monetization Strategies**
    - In-app purchases and subscriptions
    - Mobile ads (AdMob, Facebook Audience Network)
    - Affiliate marketing and sponsorships
- **Legal and Compliance Considerations**
    - An introduction to the Data Protection Act
    - DPA and data privacy
    - Terms of Service & Privacy Policies
    - Intellectual property protection for mobile apps

Capstone Project: Full Mobile Application Development

As the final project in this course, students will develop a **fully functional mobile application** based on a real-world use case. The application should demonstrate innovative features, and apply the knowledge gained throughout the course.

PROJECT SCOPE

Students can **work individually or in a team of up to three (3) members**.

- **Teams must apply in advance for approval** before working on a group project.
- Each project should focus on solving a problem within one of the following areas:

1. Health & Medical

- **Emergency Response App** – A platform to connect first responders with civilians, offering **real-time location tracking** and emergency medical tips.
- **Medication Reminder & Tracker** – A mobile app that **sends reminders for medication intake**, logs symptoms, and integrates with pharmacies.
- **Telemedicine & Health Consultation** – A **secure** video/audio-based platform for patients to consult doctors remotely.
- **Mental Health Support App** – **AI-powered** chatbot for mental health counselling, with **mood tracking** and self-care resources.
- **First Aid & CPR Training Guide** – An interactive mobile guide using videos, quizzes, and **AR-based simulations** to train users.

2. Religion

- **Daily Devotional & Prayer App** – A platform that delivers **daily scripture, prayer guides, and devotionals** for users.
- **Church & Worship App** – A mobile app for **livestreaming sermons, event management, and Bible study groups**.
- **Faith-Based Meditation & Affirmations** – A relaxation and mindfulness app with **spiritual affirmations and guided prayers**.
- **Community Outreach & Volunteer Platform** – A **social network** for churches to **coordinate charity events and outreach programs**.

3. Education

- **E-Learning & Skill Development App** – A mobile learning platform with **video tutorials, quizzes, and progress tracking**.
- **Interactive Language Learning** – An **AI-powered** language tutor for teaching **English, Spanish, or Patois translation**.
- **Specialized Exam Preparation** – A study app that helps students prepare for **CXC, CAPE, or military entrance exams**.
- **Police Training & Case Study App** – A digital library containing **case studies, crime scene simulations, and JCF regulations**.

4. Small Business

- **Point-of-Sale (POS) System for Mobile** – A **small business POS solution with inventory tracking and digital receipts**.
- **Service Booking & Appointment Scheduling** – A business management app for **barbers, salons, car rental business, and home repair services**.
- **E-commerce Storefront App** – A **mobile shopping platform** with payment integration and delivery tracking.
- **Local Agro-Business Marketplace** – A **farmers' market app** for connecting local farmers to consumers and businesses.
- **Business Expense & Financial Tracker** – A tool that helps small business owners track income, expenses, and taxes.

5. Security

- **Personal Safety & SOS Alert App** – A security app with **GPS tracking, emergency alerts,** and direct connection to law enforcement.
- **Smart Neighborhood Watch** – A crowdsourced crime-reporting app where communities can report suspicious activity and track local incidents.
- **AI-based Threat Detection** – An **image recognition app** that scans surroundings for **potential security risks** (useful for law enforcement).
- **Digital Evidence & Report Filing** – A secure document submission platform for law enforcement officers to store and submit reports.
- **Disaster & Emergency Preparedness App** – A hurricane, flood, and disaster response system with AI-powered risk assessments.

Each project must include the following components:

1. UI/UX Design

- Implement **modern, intuitive, and user-friendly interfaces**.
- Follow best practices for **mobile app usability and accessibility**.
- Create wireframes or prototypes before development.

2. Backend API Integration

- Use **Spring Boot, Firebase, or another suitable backend** for data storage and retrieval.
- Implement secure **RESTful APIs** for communication between the mobile app and the backend.
- Integrate **database solutions** (e.g., MySQL, Firebase Firestore, SQLite) based on app requirements.

3. Authentication & Security

- Implement **secure user authentication** using JWT, OAuth2, or Firebase Authentication.
- Apply **role-based access control (RBAC)** where necessary.
- Ensure **data encryption** and proper security measures to protect user information.

4. Deployment & Testing

- Conduct **unit and integration testing** to ensure app reliability.
- Use **Flutter test framework**, Espresso (for Android), or manual testing for validation.
- Optimize **performance, scalability, and responsiveness**.
- Deploy the final application to the **Google Play Store** or **App Store**, following best practices for app publishing.

Final Presentation & Assessment

- Each student (or team) will **present their application** demonstrating functionality, security measures, and performance optimizations.
- The application should include **proper documentation**, including system design, API references, and setup instructions.
- Final projects will be **evaluated based on innovation, usability, and technical execution**.

Project Timeline

- Students will have 1 extra month (30 days) to complete the capstone project at the end of the course.
- **Early start is encouraged**—students are advised to begin planning and working on their projects as soon as they have enough foundational knowledge.
- Team project applications must be submitted in advance for approval.